EEE3096S Lecture 04

Quick Scenario: Designing a Golden Measure

The Problem:

Imagine you are building a low-power, real-time embedded system (e.g., a wearable fitness tracker or a mobile robot sensor) that needs to calculate the Euclidean distance d between two 2D points, (x_1, y_1) and (x_2, y_2), thousands of times per second:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In your final embedded C code, to save CPU cycles and power, you plan to use an optimized approximation (such as a fast fixed-point algorithm or the Alpha Max + Beta Min method).

The Question for You:

Before writing or optimizing your fast embedded C algorithm, **what would serve as your Golden Measure for this problem, and how would you implement/verify it?**

(Take 30 seconds to think about it, then flip the page over!) 🔄

EEE3096S Lecture 04

Quick Scenario: Designing a Golden Measure

The Problem:

Imagine you are building a low-power, real-time embedded system (e.g., a wearable fitness tracker or a mobile robot sensor) that needs to calculate the Euclidean distance d between two 2D points, (x_1, y_1) and (x_2, y_2), thousands of times per second:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

In your final embedded C code, to save CPU cycles and power, you plan to use an optimized approximation (such as a fast fixed-point algorithm or the Alpha Max + Beta Min method).

The Question for You:

Before writing or optimizing your fast embedded C algorithm, **what would serve as your Golden Measure for this problem, and how would you implement/verify it?**

(Take 30 seconds to think about it, then flip the page over!) 🔄

EEE3096S Lecture 04

The Solution & Takeaway

The Golden Measure:

1. **High-Precision Implementation:** A simple MATLAB/Python script or double-precision C function using standard library calls (sqrt(), pow()).
2. **Verification:** Test it using known geometric identities (e.g., standard 3-4-5 right triangles) or exhaustive testing against exact mathematical values.

Why is this a Golden Measure?

- **It doesn't care about speed or memory:** It might run slowly or use heavy double-precision floats that your microcontroller can't handle efficiently.
- **It prioritizes 100% numerical correctness:** It gives you a trusted baseline output.

How you use it in practice:

When you build your fast, fixed-point C algorithm on the micro, you compare its outputs against the Golden Measure dataset to quantify the exact error margin (e.g., *"Our fast algorithm runs 10× faster but has a maximum deviation of 0.4% relative to the Golden Measure"*).

EEE3096S Lecture 04

The Solution & Takeaway

The Golden Measure:

3. **High-Precision Implementation:** A simple MATLAB/Python script or double-precision C function using standard library calls (sqrt(), pow()).
4. **Verification:** Test it using known geometric identities (e.g., standard 3-4-5 right triangles) or exhaustive testing against exact mathematical values.

Why is this a Golden Measure?

- **It doesn't care about speed or memory:** It might run slowly or use heavy double-precision floats that your microcontroller can't handle efficiently.
- **It prioritizes 100% numerical correctness:** It gives you a trusted baseline output.

How you use it in practice:

When you build your fast, fixed-point C algorithm on the micro, you compare its outputs against the Golden Measure dataset to quantify the exact error margin (e.g., *"Our fast algorithm runs 10× faster but has a maximum deviation of 0.4% relative to the Golden Measure"*).